



Esercitazione 4

Discrete Fourier Transform



Introduzione

L'esercitazione introduce il concetto di **Trasformata di Fourier Discreta (DFT)** come strumento fondamentale per analizzare segnali e immagini digitali nel dominio della frequenza. L'obiettivo principale è verificare le proprietà teoriche della DFT e della sua inversa (IDFT), comprendere il ruolo delle basi ortonormali di Fourier e sperimentare le principali applicazioni pratiche legate all'analisi spettrale.

In particolare, si studiano:

- la costruzione e le proprietà della base di Fourier discreta;
- la conservazione dell'energia (identità di Parseval);
- gli effetti di traslazione e modulazione nel tempo e in frequenza;
- il legame tra convoluzione e prodotto in frequenza.

Obiettivo finale: acquisire strumenti sia teorici che computazionali per rappresentare e manipolare segnali nel dominio della frequenza, evidenziando come la DFT sia la base di molte tecniche moderne di elaborazione digitale (filtraggio, analisi spettrale, compressione).



Riassunto Esercitazione 4

In questa esercitazione si è analizzata la **Discrete Fourier Transform (DFT)** e la sua implementazione numerica tramite **Python (SciPy)**.

Gli step principali svolti sono stati:

- **Definizione della DFT e IDFT:** studio delle formule, della periodicità e della ricostruzione esatta della sequenza originaria.
- **Costruzione della base di Fourier discreta:** visualizzazione degli atomi di base (parte reale e immaginaria) e confronto con la base canonica.
- **Verifica dell'ortogonalità/ortonormalità** delle basi tramite il calcolo dei prodotti scalari.
- **Parseval identity:** dimostrazione numerica della conservazione dell'energia tra dominio del tempo e della frequenza.
- **Proprietà fondamentali della DFT:** linearità, traslazioni, modulazioni e teorema della convoluzione, verificate su sequenze test.
- **Implementazione pratica in Python:** utilizzo dei moduli `scipy.fftpack` e `scipy.signal` per calcolare DFT e IDFT e svolgere analisi spettrali.

Obiettivo raggiunto: mostrare come la DFT fornisca una rappresentazione compatta e strutturata dei segnali, utile sia per l'analisi teorica sia per applicazioni numeriche e ingegneristiche.



Trasformazione di Fourier discreta (DFT)

La **DFT** trasforma una sequenza discreta $x = [x_0, \dots, x_{N-1}]$ in una rappresentazione in frequenza X_k , definita come:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-j2\pi nk/N}, \quad k = 0, \dots, N-1$$

- **Periodicità:** $X_{k+N} = X_k$. Talvolta si usano indici simmetrici attorno a 0.
- **IDFT:** ricostruisce la sequenza originale tramite

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{j2\pi nk/N}.$$

- **Fourier basis:** le funzioni $\phi_n^{(k)} = \frac{1}{\sqrt{N}} e^{j2\pi kn/N}$ formano una base ortonormale di $\mathbb{C}^N$.
- **Parseval:** l'energia è conservata tra dominio del tempo e della frequenza

$$\sum_{n=0}^{N-1} |x_n|^2 = \sum_{k=0}^{N-1} |X_k|^2.$$

- **Proprietà principali:**
 - Linearità
 - Traslazione nel tempo $\leftrightarrow$ moltiplicazione in frequenza
 - Modulazione $\leftrightarrow$ traslazione in frequenza
 - Convoluzione $\leftrightarrow$ prodotto in frequenza
- **Analisi spettrale in Python:** disponibili i moduli `fftpack` e `signal` in SciPy (`f.fft`, `f.ifft`).

🔑 Punti chiave: la DFT permette di passare da una descrizione nel dominio del tempo a una nel dominio delle frequenze, con proprietà utili per analisi, filtraggio e convoluzione.

```
In [2]: import numpy as np
import matplotlib.pyplot as plt
import scipy.fft as fft
```



Esercizio 1

Calcolare attraverso la definizione la DFT dei seguenti vettori:

- `v1 = [2, 3]`

- `v2 = [2, 4, 5]`
- `v3 = [3, 4, 5, 6]`

Verificare con Python la correttezza utilizzando `scipy.fft.fft`.

```
In [6]: # Vettori di esempio
x1 = np.array([2, 3])
x2 = np.array([2, 4, 5])
x3 = np.array([3, 4, 5, 6])

# Lista di vettori da testare
vettori = [(x1, "v1 = [2, 3]"), (x2, "v2 = [2, 4, 5]"), (x3, "v3 = [3, 4, 5, 6]")]

for x, label in vettori:
    print(f"\n ♦ Test su {label}")

    # Calcolo DFT con scipy
    X_lib = fft.fft(x)

    # Calcolo DFT con funzione definita
    X_custom = dft(x)

    # Calcolo IDFT per verifica ricostruzione
    x_rec = idft(X_custom)

    # Confronto tra trasformate
    dft_diff = np.max(np.abs(X_custom - X_lib))
    print(f"   ✔ Errore max tra DFT custom e scipy: {dft_diff:.2e}")

    # Confronto tra segnale originale e ricostruito
    rec_diff = np.max(np.abs(x_rec - x))
    print(f"   ☑ Errore max ricostruzione IDFT: {rec_diff:.2e}")
```

- ♦ Test su v1 = [2, 3]
 - ✔ Errore max tra DFT custom e scipy: 3.67e-16
 - ☑ Errore max ricostruzione IDFT: 1.84e-16
- ♦ Test su v2 = [2, 4, 5]
 - ✔ Errore max tra DFT custom e scipy: 3.64e-15
 - ☑ Errore max ricostruzione IDFT: 1.18e-15
- ♦ Test su v3 = [3, 4, 5, 6]
 - ✔ Errore max tra DFT custom e scipy: 3.02e-15
 - ☑ Errore max ricostruzione IDFT: 1.14e-15

COMMENTO.

Per ciascun vettore (`v1=[2,3]`, `v2=[2,4,5]`, `v3=[3,4,5,6]`) la DFT calcolata "a definizione" coincide con `scipy.fft.fft` con errore massimo nell'ordine di 10^{-15} - 10^{-16} (floating point). L'IDFT ricostruisce il segnale originale con errore massimo dello stesso ordine, confermando la **correttezza dell'implementazione** e la **biunivocità** DFT↔IDFT.

Take-home message: implementazione corretta (allineata a SciPy), ricostruzione perfetta a precisione numerica; le differenze osservate sono solo **round-off**.

🚀 Esercizio 2

Scrivere una funzione che calcoli la DFT di un vettore.

Verificare la correttezza della funzione confrontando con `scipy.fft.fft`.

```
In [4]: ### -----
### Esercizio 2 - Funzione per calcolare la DFT
### -----

def dft(x):
    N = len(x)
    X = np.zeros(N, dtype=complex)
    for k in range(N):
        for n in range(N):
            X[k] += x[n] * np.exp(-2j * np.pi * k * n / N)
    return X
```

🚀 Esercizio 3

Scrivere una funzione che calcoli la DFT inversa di un vettore di coefficienti.

Verificare la correttezza confrontando con `scipy.fft.ifft`.

```
In [5]: ### -----
### Esercizio 3 - Funzione per calcolare la IDFT
### -----

def idft(X):
    N = len(X)
    x = np.zeros(N, dtype=complex)
    for n in range(N):
        for k in range(N):
            x[n] += X[k] * np.exp(2j * np.pi * k * n / N)
    return x / N
```

🚀 Esercizio 4

Sia $X = [0, 1, 2, 3, 4]$ la DFT di un certo vettore x .

Quale sarà la DFT del vettore $y(n) = e^{i4\pi n/5}x(n)$?

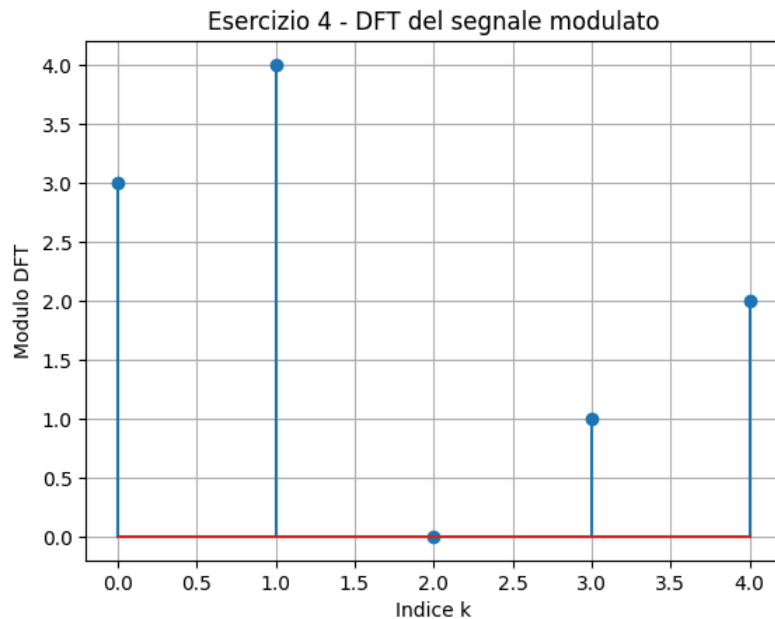
Rispondere prima utilizzando le proprietà della DFT e poi verificare con il codice sviluppato.

In [7]:

```
### -----  
### Esercizio 4 - Effetto della modulazione complessa  
### -----  
  
# Supponiamo che X sia la DFT di un segnale x  
X = np.array([0, 1, 2, 3, 4], dtype=complex)  
  
# Ricostruiamo x tramite IDFT  
x = idft(X)  
  
# Definizione della modulazione:  $y(n) = \exp(i * 4\pi * n / 5) * x(n)$   
N = len(x)  
y = np.zeros(N, dtype=complex)  
for n in range(N):  
    y[n] = np.exp(1j * 4 * np.pi * n / N) * x[n]  
  
# Calcoliamo la DFT del segnale modulato y  
Y = dft(y)  
  
# Risultato atteso teorico:  
# DFT{  $e^{i2\pi kn/N} * x(n)$  } = shift ciclico della DFT originale  
# Qui: moltiplicare per  $e^{i4\pi n/5}$   $\Rightarrow$  shift di 2 posizioni (per  $N = 5$ )  
  
print("DFT originale X:", X)  
print("DFT del segnale modulato Y:", np.round(Y, 2))  
  
# Plot del modulo della DFT  
plt.stem(np.abs(Y))  
plt.ylabel('Modulo DFT')  
plt.xlabel('Indice k')  
plt.title('Esercizio 4 - DFT del segnale modulato')  
plt.grid(True)  
plt.show()
```

DFT originale X: [0.+0.j 1.+0.j 2.+0.j 3.+0.j 4.+0.j]

DFT del segnale modulato Y: [3.-0.j 4.-0.j 0.+0.j 1.+0.j 2.+0.j]



COMMENTO.

Modulando $x(n)$ con $e^{j\frac{4\pi}{5}n}$ (cioè con passo 2 su $N = 5$) lo **spettro** viene **traslato circolarmente** di 2 posizioni: partendo da $X = [0, 1, 2, 3, 4]$ si osserva $Y = [3, 4, 0, 1, 2]$ (esattamente uno **shift +2**). Nel grafico del **modulo di Y** i picchi sono gli stessi di X , semplicemente **riordinati** (traslati) sui nuovi indici.

Interpretazione: la proprietà di modulazione DFT dice che moltiplicare nel tempo per $e^{i2\pi k_0 n/N}$ produce uno **shift di k_0** nello spettro. Qui $k_0 = 2$.

Take-home message: **modulazione nel tempo = traslazione in frequenza** (shift circolare degli indici della DFT).

✦ Esercizio 5

Considerare i segnali:

- $2018 * \sin(2 k \pi t)$, $k = 1, 2, 3$;
- $\cos(2 k \pi t)$, $k = 1, 2, 3$;
- $\cos(2 \pi t) + \cos(6 \pi t) + \cos(10 \pi t)$;
- $e^{j2 \pi i t}$;

dove t è un vettore equispaziato di 50 punti in $[0, 1]$.

Calcolare la DFT usando la funzione creata e fare un grafico del **modulo** della trasformata.

Spiegare i risultati alla luce delle proprietà della DFT.

In [8]:

```
# -----
# Esercizio 5 - Analisi spettrale segnali
# -----

# Vettore temporale equispaziato
t = np.arange(50) / 50
N = len(t)

### -----
### (a) Segnali sinusoidali
### -----

a1 = 2018 * np.sin(2 * np.pi * t)
a2 = 2018 * np.sin(2 * np.pi * 2 * t)
a3 = 2018 * np.sin(2 * np.pi * 3 * t)

A1 = np.abs(dft(a1))
A2 = np.abs(dft(a2))
A3 = np.abs(dft(a3))

plt.figure()
plt.plot(np.arange(N), A1, 'r--', label='k=1')
plt.plot(np.arange(N), A2, 'b--', label='k=2')
plt.plot(np.arange(N), A3, 'g--', label='k=3')
plt.title("Esercizio 5(a) - DFT di 2018*sin(2kπt)")
plt.xlabel("Indice DFT")
plt.ylabel("Modulo DFT")
plt.legend()
plt.grid(True)
plt.show()

### -----
### (b) Coseni a frequenze diverse
### -----

b1 = np.cos(2 * np.pi * t)
b2 = np.cos(2 * np.pi * 2 * t)
b3 = np.cos(2 * np.pi * 3 * t)

B1 = np.abs(dft(b1))
B2 = np.abs(dft(b2))
B3 = np.abs(dft(b3))

plt.figure()
plt.plot(np.arange(N), B1, 'r--', label='k=1')
plt.plot(np.arange(N), B2, 'b--', label='k=2')
plt.plot(np.arange(N), B3, 'g--', label='k=3')
plt.title("Esercizio 5(b) - DFT di cos(2kπt)")
plt.xlabel("Indice DFT")
plt.ylabel("Modulo DFT")
plt.legend()
plt.grid(True)
plt.show()

### -----
### (c) Somma di coseni
### -----

c = np.cos(2*np.pi*t) + np.cos(6*np.pi*t) + np.cos(10*np.pi*t)
C = np.abs(dft(c))

plt.figure()
plt.stem(C)
plt.title("Esercizio 5(c) - Somma di coseni")
plt.xlabel("Indice DFT")
plt.ylabel("Modulo DFT")
plt.grid(True)
plt.show()

### -----
### (d) Esponenziale complessa
### -----

d = np.exp(2j * np.pi * t)
D = np.abs(dft(d))

plt.figure()
plt.stem(D)
plt.title("Esercizio 5(d) - DFT di exp(2jπt)")
plt.xlabel("Indice DFT")
plt.ylabel("Modulo DFT")
plt.grid(True)
plt.show()

### -----
### (Extra) Spectral Leakage
### -----

tt = np.arange(64) / 64
bb = np.cos(2 * np.pi * 4 * tt)
BB = np.abs(dft(bb))

s = np.linspace(0, 1, 64)
```

```

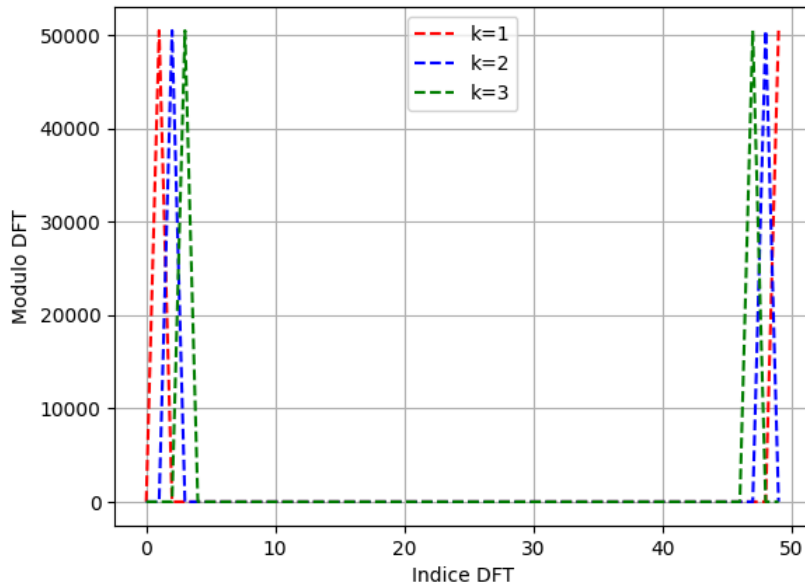
bs = np.cos(2 * np.pi * 4.063 * s)
Bs = np.abs(dft(bs))

# Segnale perfettamente periodico (frequenza intera)
plt.figure()
plt.stem(BB)
plt.title("64-point DFT, m=4 (periodico)")
plt.xlabel("Indice DFT")
plt.ylabel("Modulo DFT")
plt.grid(True)
plt.show()

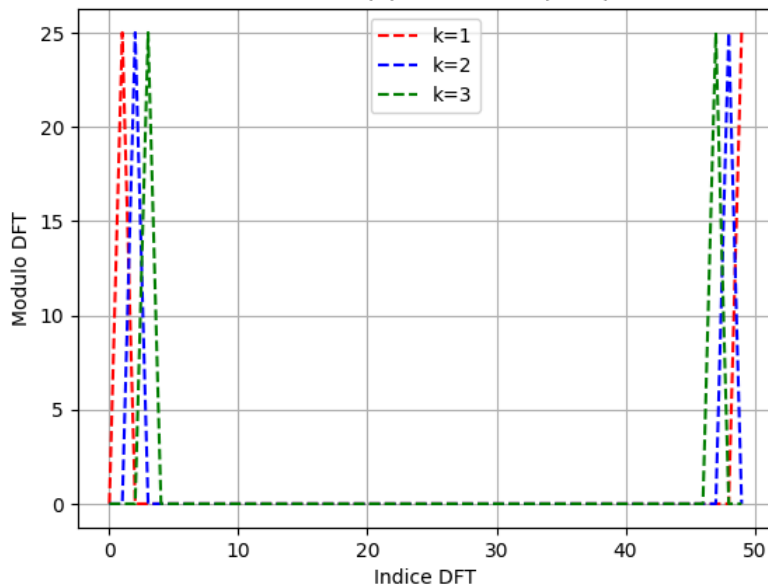
# Segnale non perfettamente periodico (spettro disperso)
plt.figure()
plt.stem(Bs)
plt.title("64-point DFT, m=4.063 (non periodico)")
plt.xlabel("Indice DFT")
plt.ylabel("Modulo DFT")
plt.grid(True)
plt.show()

```

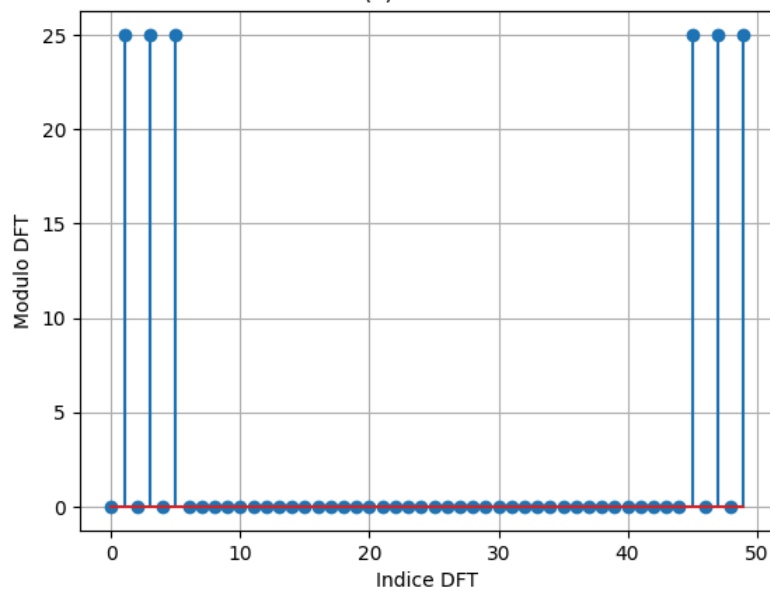
Esercizio 5(a) - DFT di $2018 \sin(2k\pi t)$



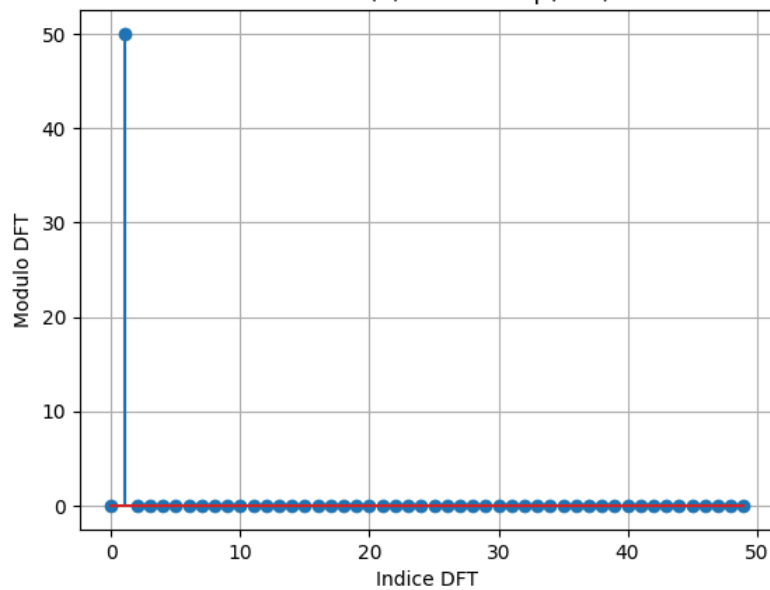
Esercizio 5(b) - DFT di $\cos(2k\pi t)$



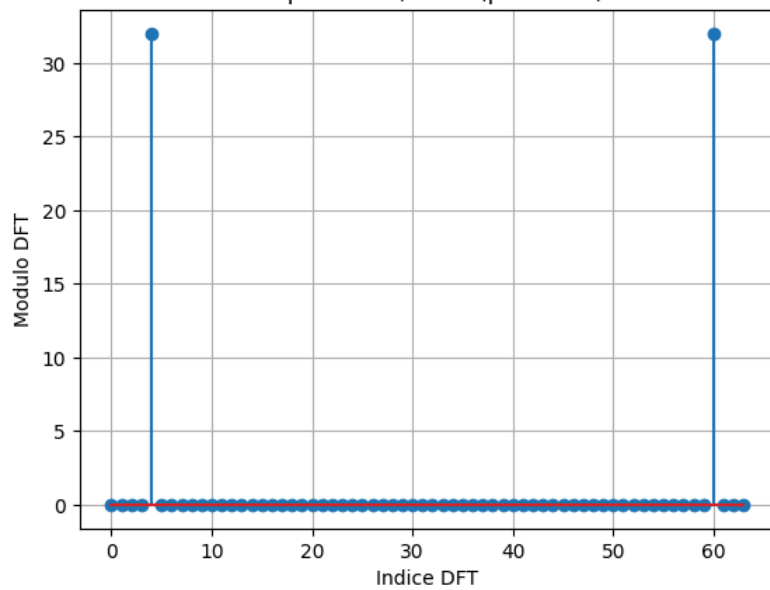
Esercizio 5(c) - Somma di coseni

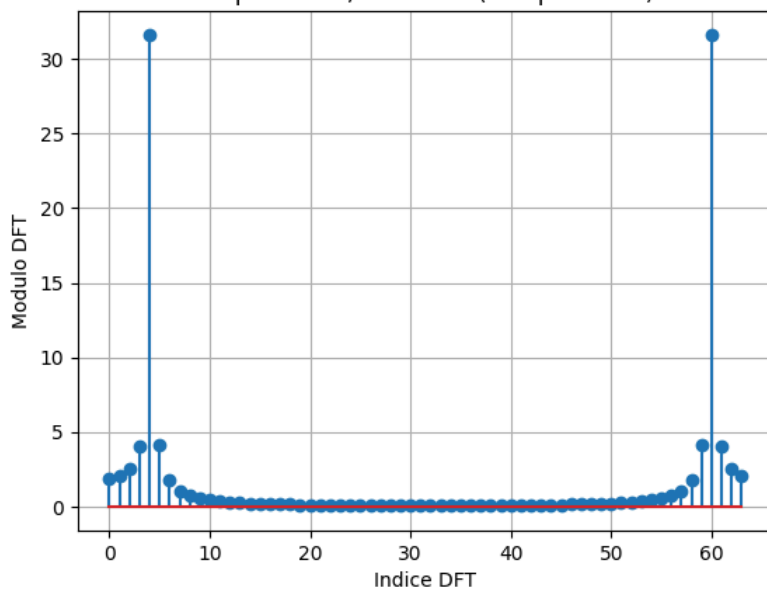


Esercizio 5(d) - DFT di $\exp(2\pi i t)$



64-point DFT, $m=4$ (periodico)



64-point DFT, $m=4.063$ (non periodico)**COMMENTO.**

(a) $2018 \cdot \sin(2\pi kt)$, $k = 1, 2, 3$ Nei grafici del **modulo DFT** compaiono **due picchi coniugati** alle frequenze k e $N - k$ (simmetria coniugata), con ampiezza scalata dal fattore 2018. All'aumentare di k i picchi si spostano sui corrispondenti indici.

(b) $\cos(2\pi kt)$, $k = 1, 2, 3$ Anche qui compaiono **due righe spettrali** a k e $N - k$. Il coseno è reale e pari $\rightarrow$ spettro reale ed **esattamente simmetrico**. Cambiando k , i picchi "migrano" ai nuovi indici, restando ben localizzati.

(c) $\cos(2\pi t) + \cos(6\pi t) + \cos(10\pi t)$ Il modulo della DFT mostra **tre coppie di picchi** ai corrispondenti indici delle frequenze componenti: 1 e $N - 1$; 3 e $N - 3$; 5 e $N - 5$. Il grafico evidenzia la **sovrapposizione lineare** in frequenza: lo spettro è la somma degli spettri dei singoli termini.

(d) $e^{2\pi i t}$ L'esponenziale complessa **positiva** genera **una sola riga** a $k = 1$ (niente coppia coniugata perché il segnale è complesso e unilaterale in frequenza). Il plot mostra un **unico picco** netto.

(Extra) **Leakage spettrale** Con $N = 64$ e $\cos(2\pi \cdot 4 t)$ (frequenza **intera** sui 64 campioni) compaiono **due righe pulite** a $k = 4$ e $k = 60$. Con frequenza **non intera** (4.063) i picchi si **spalmano** sui bin adiacenti (finestra rettangolare) generando il classico **Dirichlet/leakage** osservato nel grafico (picco centrale con **lobi laterali**).

Take-home message:

- Sinusoidi reali $\rightarrow$ **due righe** a k e $N - k$; esponenziale complessa $\rightarrow$ **una riga**.
- Somma di sinusoidi $\rightarrow$ **somma degli spettri** (picchi alle frequenze componenti).
- **Leakage** quando la frequenza **non è multiplo** di $1/N$: energia "distribuita" su più bin.

★ **Esercizio 6**

Considerare i segnali di lunghezza 50:

- $[0, 10, 0, 0, \dots, 0]$
- $[0, 10, 0, 0, \dots, 0, 10]$

Calcolare la DFT usando la funzione creata e fare un grafico del **modulo**.

Spiegare i risultati alla luce delle proprietà della DFT.

```
In [9]: # -----
# Esercizio 6 - DFT di impulsi localizzati
# -----

# Lunghezza dei segnali
N = 50
t = np.arange(N)

# Segnale a: impulso singolo
a = np.zeros(N)
a[1] = 10 # impulso in posizione 1

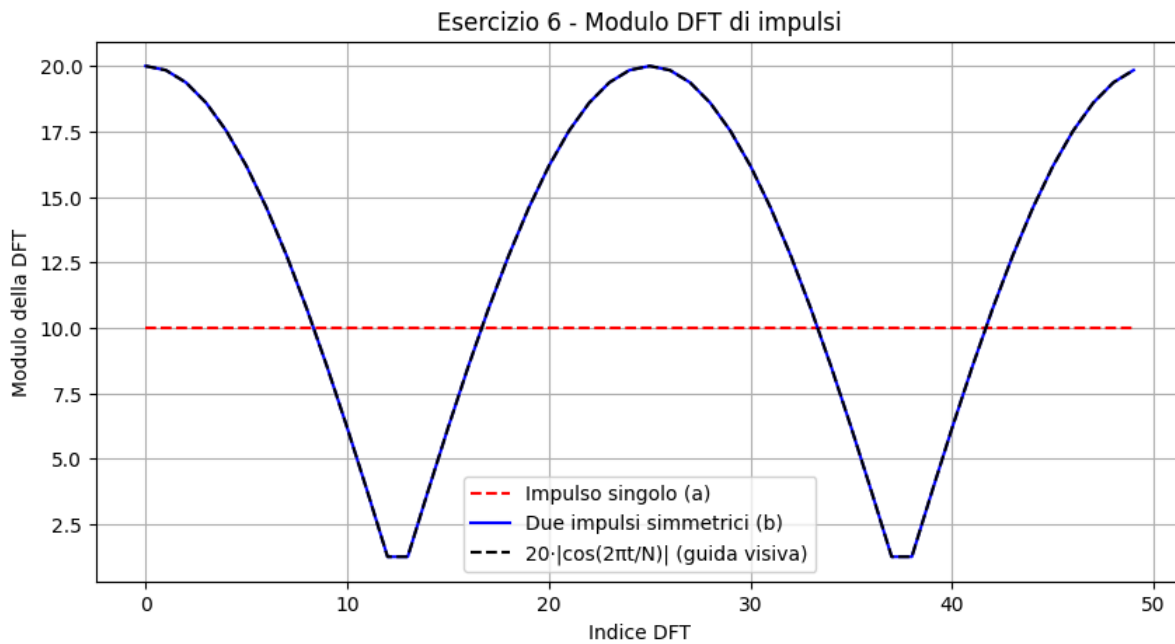
# Segnale b: due impulsi simmetrici
b = np.zeros(N)
b[1] = 10
b[-1] = 10 # impulso in posizione N-1 (equivalente a -1 per periodicità)

# Calcolo della DFT (modulo)
A = np.abs(dft(a))
B = np.abs(dft(b))

# -----
# Plot del modulo della DFT
# -----
plt.figure(figsize=(10, 5))
plt.plot(t, A, 'r--', label="Impulso singolo (a)")
```

```
plt.plot(t, B, 'b-', label="Due impulsi simmetrici (b)")
plt.plot(t, 20 * np.abs(np.cos(2 * np.pi * t / N)), 'k--', label="20·|cos(2πt/N)| (guida visiva)")

plt.title("Esercizio 6 - Modulo DFT di impulsi")
plt.xlabel("Indice DFT")
plt.ylabel("Modulo della DFT")
plt.legend()
plt.grid(True)
plt.show()
```



COMMENTO.

- **Impulso singolo** a (ampiezza 10 in posizione 1): il **modulo della DFT è piatto** (tutte le ampiezze ≈ 10). È la dualità **impulso** $\leftrightarrow$ **costante** in frequenza (lo shift nel tempo cambia solo la fase, non il modulo).
- **Due impulsi simmetrici** b (posizioni 1 e $N - 1$): il **modulo** segue esattamente la curva $20 \cdot |\cos(2\pi k/N)|$, come mostrato dal confronto con la guida visiva. Infatti $B[k] = 10 e^{-i2\pi k/N} + 10 e^{i2\pi k/N} = 20 \cos(2\pi k/N) \Rightarrow |B[k]| = 20 |\cos(\cdot)|$.

Take-home message:

- Un **impulso singolo** ha **spettro a modulo costante**.
- Due impulsi **equidistanti** generano uno **spettro cosinusoidale** (in modulo), in accordo con la somma di due esponenziali coniugate.